

GetLidar()

nom / prénom : El fares youssef

fonction réalisée : getLidarBits()

explication : Cette fonction lit les états des 6 capteurs LIDAR branchés sur les GPIO 3 à 8 de la Raspberry Pi Pico, et retourne un seul octet (uint8_t) où chaque bit représente l'état d'un capteur (0 = rien détecté, 1 = obstacle détecté).

exemple d'utilisation : Dans un while(1) on declare une variable " lidars = getLidarBits(); "

code :

```
#include <stdio.h>
#include <stdint.h>
#include "pico/stdlib.h"

#define NUM_LIDARS    6
// Masque pour GP3..GP8
#define LIDAR_MASK    ((1<<3) | (1<<4) | (1<<5) | (1<<6) | (1<<7) | (1<<8))

// Initialisation des broches en entrée
void initLidarPins(void) {
    for (int i = 0; i < NUM_LIDARS; i++) {
        gpio_init(lidar_pins[i]);          // « réveille » la broche GPx
        gpio_set_dir(lidar_pins[i], GPIO_IN); // la configure en entrée
    }
}

/// Retourne un octet dont les 6 bits de poids faible
/// correspondent à GP3..GP8
uint8_t getLidarBits(void) {
    uint32_t all = gpio_get_all();          // lit GP0..GP29 en un mot
    return (uint8_t)((all & LIDAR_MASK) >> 3);
}

int main(void) {
    stdio_init_all();          // USB-série
    initLidarPins();

    while (1) {
        uint8_t bitLidars = getLidarBits();
        printf("LIDAR bits: 0x%02X\n", bitLidars);

    }

    return 0;
}
```

```
}
```

Nom / prénom : BOURGEOIS Noa

Fonction réalisée : AfficherIhm

Explication : Récupérer les données de connexion et valeurs transmises par les différents capteurs du robot.

Exemple d'utilisation : débogage en temps réel de la viabilité des capteurs

Code :

```
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/i2c.h"

//LIDAR
#define NUM_LIDARS    6
// Masque pour GP3..GP8
#define LIDAR_MASK    ((1<<3)|(1<<4)|(1<<5)|(1<<6)|(1<<7)|(1<<8))
const uint lidar_pins[NUM_LIDARS] = {3, 4, 5, 6, 7, 8};

// Initialise les broches LIDAR en entrée
void initLidarPins(void) {
    for (int i = 0; i < NUM_LIDARS; i++) {
        gpio_init(lidar_pins[i]);
        gpio_set_dir(lidar_pins[i], GPIO_IN);
    }
}

// Lit l'état des 6 capteurs LIDAR
uint8_t getLidarBits(void) {
    uint32_t all = gpio_get_all(); // lecture
    return (uint8_t)((all & LIDAR_MASK) >> 3); // applique masque + décale de 3
}

// Fonctions simulées
int getIrValue(uint8_t address) { return 5; }
bool getCapteurLigne(uint8_t address) { return 0; }
bool detectionBalisePixy(uint8_t address) { return 0; }
int getPixy(uint8_t address) { return 0; }
bool i2cinit(uint8_t address) { return 1; }

void sendI2CByte(uint8_t device_addr, uint8_t reg_addr, uint8_t value) {
    uint8_t buf[2] = {reg_addr, value};
    i2c_write_blocking(i2c1, device_addr, buf, 2, false);
}

int main() {
    stdio_init_all();
    initLidarPins(); //Lidar initialisation

    // I2C1 sur GPIO18 (SDA) et GPIO19 (SCL)
    i2c_init(i2c1, 100 * 1000);
```

```

gpio_set_function(18, GPIO_FUNC_I2C); // SDA
gpio_set_function(19, GPIO_FUNC_I2C); // SCL
gpio_pull_up(18);
gpio_pull_up(19);

while (true) {
    int recepteurIr = getIrValue(0x26);

    bool capteurLigne1 = getCapteurLigne(0x2A);
    bool capteurLigne2 = getCapteurLigne(0x2B);

    bool pixyDetected = detectionBalisePixy(0x2C);
    int direction = getPixy(0x2D);

    // Nouvelle lecture réelle des lidars
    uint8_t lidarBits = getLidarBits();
    sendI2CByte(0x10, 0x20, lidarBits); // envoie du registre 0x20 (octet de bits)

    bool etatMPU = i2cinit(0x2E);
    bool etatRecepteurIr1 = i2cinit(0x2F);
    bool etatRecepteurIr2 = i2cinit(0x30);
    bool etatRecepteurIr3 = i2cinit(0x31);
    bool etatCarteCelebration = i2cinit(0x32);

    sendI2CByte(0x10, 0x2C, pixyDetected);
    sendI2CByte(0x10, 0x2D, direction);

    sendI2CByte(0x10, 0x2A, capteurLigne1);
    sendI2CByte(0x10, 0x2B, capteurLigne2);

    sendI2CByte(0x10, 0x26, recepteurIr);

    sendI2CByte(0x10, 0x2E, etatMPU);
    sendI2CByte(0x10, 0x2F, etatRecepteurIr1);
    sendI2CByte(0x10, 0x30, etatRecepteurIr2);
    sendI2CByte(0x10, 0x31, etatRecepteurIr3);
    sendI2CByte(0x10, 0x32, etatCarteCelebration);
    sendI2CByte(0x10, 0x33, 1);
    sleep_ms(20);
}
}

```

Nom/Prénom : Tristan Stevenin

Fonction réalisée : cartelhm

Explication : Récupérer les données envoyées par la pico et les afficher sur un écran tactile.

Exemple d'utilisation : débogage en temps réel de la viabilité des capteurs

```
#include "U8glib.h"
#include <Wire.h>

// Données reçues
float signalIR = 0;
bool pixyDetected = false;
int pixyDirection = 0;
bool capteurLigne1 = false;
bool capteurLigne2 = false;
volatile bool lidar[6] = { false, false, false, false, false, false };
byte lidarStatus = 0;
bool etatMPU = false;
bool etatReceptrice1 = false;
bool etatReceptrice2 = false;
bool etatReceptrice3 = false;
bool etatCelebration = false;
volatile bool refresh = 1;

// Écran graphique
U8GLIB_KS0108_128 u8g(7, 6, 5, 4, 3, 2, 1, 0, 11, 8, 9, 10, 12);

// Menu principal
int menuIndex = 0;
const int menuSize = 5;
const char *menuItems[] = {
    "LIDAR",
    "Capteur IR",
    "Cam Pixy",
    "Capteur Ligne",
    "Etat I2C"
};

// Écran tactile
const int haut = A1;
const int bas = A2;
const int droite = A3;
const int gauche = A0;

// Lecture tactile
unsigned int posTouchY() {
    pinMode(droite, OUTPUT);
    pinMode(gauche, OUTPUT);
    digitalWrite(droite, 0);
    digitalWrite(gauche, 1);
    delay(1);
    int pos = analogRead(haut);
    pinMode(droite, INPUT_PULLUP);
    pinMode(gauche, INPUT_PULLUP);
    return pos;
}

unsigned int posTouchX() {
    pinMode(haut, OUTPUT);
    pinMode(bas, OUTPUT);
    digitalWrite(haut, 0);
    digitalWrite(bas, 1);
    delay(1);
    int pos = analogRead(droite);
    pinMode(haut, INPUT_PULLUP);
    pinMode(bas, INPUT_PULLUP);
    return pos;
}

// Affichage menu principal
void drawMenu() {
    u8g.firstPage();
    do {
        u8g.setFont(u8g_font_6x10);
        for (int i = 0; i < menuSize; i++) {
            if (i == menuIndex) {
                u8g.setColorIndex(1);
                u8g.drawBox(0, i * 12, 98, 12);
                u8g.setColorIndex(0);
            } else {
                u8g.setColorIndex(1);
            }
            u8g.drawStr(20, i * 12 + 10, menuItems[i]);
        }
        u8g.setColorIndex(1);
        u8g.drawTriangle(105, 50, 115, 60, 125, 50); // Bas
        u8g.drawTriangle(105, 13, 115, 3, 125, 13); // Haut
        u8g.drawBox(0, 0, 15, 60); // Validation gauche
    } while (u8g.nextPage());
}
```

```

    } while (u8g.nextPage());
}

// Sous-menus
void drawSubMenu() {
    u8g.firstPage();
    do {
        u8g.setFont(u8g_font_6x10);
        u8g.drawStr(10, 10, menuItems[menuIndex]);

        switch (menuIndex) {
            case 0: // LIDAR
                u8g.setFont(u8g_font_5x8);
                for (int i = 0; i < 6; i++) {
                    int x = (i < 3) ? 10 : 70;
                    int y = 20 + (i % 3) * 10;
                    u8g.setPrintPos(x, y);
                    u8g.print("Lidar ");
                    u8g.print(i + 1);
                    u8g.print(": ");
                    u8g.print(lidar[i]);
                }
                break;

            case 1: // Signal IR

                u8g.setPrintPos(10, 25);
                u8g.print("Valeur IR: ");
                u8g.print(signalIR, 1);

                break;

            case 2: // Cam Pixy pixyDirectiond
                u8g.drawStr(10, 25, "Detection: ");
                u8g.setPrintPos(75, 25);
                u8g.print(pixyDetected);
                u8g.drawStr(10, 40, "Direction: ");
                u8g.setPrintPos(75, 40);
                u8g.print(pixyDirection);
                break;

            case 3: // Capteur Ligne
                u8g.drawStr(10, 25, "QRE1113 N1: ");
                u8g.setPrintPos(90, 25);
                u8g.print(capteurLigne1);
                u8g.drawStr(10, 40, "QRE1113 N2: ");
                u8g.setPrintPos(90, 40);
                u8g.print(capteurLigne2);
                break;

            case 4: // Etat I2C
                u8g.setFont(u8g_font_5x8);
                u8g.setPrintPos(5, 25);
                u8g.print("IR 1: ");
                u8g.print(etatReceptrice3);

                u8g.setPrintPos(5, 35);
                u8g.print("IR 2: ");
                u8g.print(etatReceptrice1);

                u8g.setPrintPos(5, 45);
                u8g.print("IR 3: ");
                u8g.print(etatReceptrice2);

                u8g.setPrintPos(50, 25);
                u8g.print("MPU: ");
                u8g.print(etatMPU);

                u8g.setPrintPos(50, 35);
                u8g.print("Celebration: ");
                u8g.print(etatCelebration);
                break;
        }

        u8g.drawBox(10, 50, 50, 12);
        u8g.setColorIndex(0);
        u8g.drawStr(15, 60, "Retour");
        u8g.setColorIndex(1);
    } while (u8g.nextPage());
}

// Navigation menu
void updateMenu() {
    unsigned int x = posTouchX();
    unsigned int y = posTouchY();
    if (y > 700 && x < 450 && y < 1000) {
        menuIndex--;
        if (menuIndex < 0) menuIndex = menuSize - 1;
        delay(200);
    }
    if (y > 700 && x > 500 && y < 1000 && x < 1000) {
        menuIndex++;
        if (menuIndex >= menuSize) menuIndex = 0;
        delay(200);
    }
}

bool detectRetour() {
    unsigned int x = posTouchX();
    unsigned int y = posTouchY();
    return (y < 500 && x > 550 && x < 1000);
}

// Nouvelle gestion des données reçues
volatile bool shouldUpdate = false;

```

```

void receiveEvent(int numBytes) {
    while (Wire.available() >= 2) {
        refresh = true;
        byte reg = Wire.read();
        byte val = Wire.read();
        while (Wire.available()) Wire.read();
        switch (reg) {
            case 0x20:
                lidarStatus = val; // Valeur lidar sur 6 bits, 1 bit par lidar
                lidar[0] = (lidarStatus >> 0) & 0x01;
                lidar[1] = (lidarStatus >> 1) & 0x01;
                lidar[2] = (lidarStatus >> 2) & 0x01;
                lidar[3] = (lidarStatus >> 3) & 0x01;
                lidar[4] = (lidarStatus >> 4) & 0x01;
                lidar[5] = (lidarStatus >> 5) & 0x01;
                break;
            case 0x26: signalIR = val; break;
            case 0x2A: capteurLigne1 = val; break;
            case 0x2B: capteurLigne2 = val; break;
            case 0x2C: pixyDetected = val; break;
            case 0x2D: pixyDirection = val; break;
            case 0x2E: etatMPU = val; break;
            case 0x2F: etatReceptrice1 = val; break;
            case 0x30: etatReceptrice2 = val; break;
            case 0x31: etatReceptrice3 = val; break;
            case 0x32: etatCelebration = val; break;
            case 0x33: refresh = val; break;
        }
    }
}

void setup() {
    Wire.begin(0x10); // Esclave I2C
    Wire.onReceive(receiveEvent);
}

void loop() {
    updateMenu();
    drawMenu();

    if (posTouchY() < 500 && posTouchX() < 500) {
        while (true) {
            if(refresh==1){ //Limite l'actualisation de l'ecran uniquement quand on reçoit de nouvelles valeurs
                refresh = 0;
                drawSubMenu();
            }
            if (detectRetour()) break;
        }
    }
}

```

nom / prénom : ELIAS BELMELIANI
fonction réalisée : fonction celebration
explication :

exemple d'utilisation :

```

#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/i2c.h"

```

```
// === Définition I2C ===
#define I2C_PORT i2c0
#define I2C_SDA 4
#define I2C_SCL 5
#define SLAVE_ADDRESS 0x08 // Adresse de l'ATtiny

// === Fonction d'envoi de commande ===
void sendCommand(uint8_t command) {
    int result = i2c_write_blocking(I2C_PORT, SLAVE_ADDRESS, &command, 1, false);
    if (result < 0) {
        printf("Erreur d'envoi de la commande 0x%02X\n", command);
    } else {
        printf("Commande 0x%02X envoyée (%d octets)\n", command, result);
    }
}

int main() {
    stdio_init_all();

    // Initialisation I2C à 400kHz
    i2c_init(I2C_PORT, 100 * 1000);
    gpio_set_function(I2C_SDA, GPIO_FUNC_I2C);
    gpio_set_function(I2C_SCL, GPIO_FUNC_I2C);
    gpio_pull_up(I2C_SDA);
    gpio_pull_up(I2C_SCL);

    sleep_ms(2000); // Petit délai au démarrage

    while (true) {
        sendCommand(0x01); // Commande LED
        sleep_ms(2000);

        sendCommand(0x02); // Commande DFPlayer
        sleep_ms(2000);

        sendCommand(0x03); // Commande Servo
        sleep_ms(2000);
    }
}
```

codes :

Code coté Arduino (ATtiny841) pour la carte celebration :

```
#define F_CPU 8000000UL
#include <avr/io.h>
```

```

#include <util/delay.h>
#include <avr/interrupt.h>²
#include <SoftwareSerial.h>
#include <DFRobotDFPlayerMini.h>
#include <Wire.h>
#include <FastLED.h>

#define I2C_ADDRESS 0x08
#define RX_PIN 2          // PA1 - DFPlayer RX
#define TX_PIN 1          // PA0 - DFPlayer TX
#define LED_PIN 7         // Pin LED WS2812
#define SERVO_PIN PB2     // Servo Pin
#define NUM_LEDS 1

CRGB leds[NUM_LEDS];
volatile uint8_t receivedData = 0;
volatile bool newValue = false; // Initialisation à false
SoftwareSerial mySerial(RX_PIN, TX_PIN);
DFRobotDFPlayerMini myDFPlayer;

void receiveEvent(int howMany) {
    Serial.println("Donnée reçue"); // Affiche un message lorsque des données sont
reçues
    while (Wire.available()) {
        receivedData = Wire.read();
        newValue = true; // Marque la donnée comme reçue
        Serial.print("Donnée lue: ");
        Serial.println(receivedData, HEX); // Affiche la donnée reçue
    }
}

void initPins() {
    DDRA |= (1 << LED_PIN); // WS2812 LED
    DDRB |= (1 << SERVO_PIN); // Servo
}

void playAudio() {
    mySerial.begin(9600);
    delay(1000);
    if (!myDFPlayer.begin(mySerial)) {
        while (true); // Blocage si erreur
    }
    myDFPlayer.volume(10);
    myDFPlayer.play(1); // "0001.mp3"
}

void animateLED(int colorChoice) {
    if (colorChoice == 1) {

```



```

    leds[0] = CRGB(12, 0, 0); // Rouge
}
else if (colorChoice == 2) {
    leds[0] = CRGB(0, 12, 0); // Vert
}
else if (colorChoice == 3) {
    leds[0] = CRGB(0, 0, 12); // Bleu
} else {
    leds[0] = CRGB(12, 12, 12); // Gris par défaut si choix invalide
}

FastLED.show(); // Applique la couleur
}

void setup() {
    FastLED.addLeds<WS2812, LED_PIN, GRB>(leds, NUM_LEDS);
    Serial.begin(9600); // Initialiser la communication série pour le débogage
    mySerial.begin(9600); // Initialisation de la communication série pour DFPlayer
    Wire.begin(I2C_ADDRESS);
    Wire.onReceive(receiveEvent);
    sei();
    initPins();
    _delay_ms(1000);
}

void loop() {
    if (newValue) { // Vérifie si une nouvelle donnée a été reçue
        Serial.print("Donnée reçue: ");
        Serial.println(receivedData, HEX); // Affiche la donnée reçue en hexadécimal

        // Change la couleur de la LED en fonction de la donnée reçue
        if (receivedData == 1) {
            animateLED(1);
            Serial.println("1 :");
        } else if (receivedData == 2) {
            animateLED(2);
            Serial.println("2 :");
        } else if (receivedData == 3) {
            animateLED(3);
            Serial.println("3 :");
        } else {
            Serial.println("Autre :");
            animateLED(0); // Gris par défaut
        }

        newValue = false; // Réinitialise newValue pour ne pas répéter indéfiniment
    }
}

```

nom / prénom :Mezdaoui Samia

fonction réalisée **InitLidar/getlidar**

explication : Ce code lit en une seule opération l'état de six broches GPIO (GP3 à GP8) grâce à `gpio_get_all()`, puis isole et décale ces bits pour obtenir un octet compact représentant tous les capteurs LIDAR.

exemple d'utilisation

code

```
#include <stdio.h>
#include <stdint.h>
#include "pico/stdlib.h"

#define NUM_LIDARS    6
// Masque pour GP3..GP8
#define LIDAR_MASK    ((1<<3) | (1<<4) | (1<<5) | (1<<6) | (1<<7) | (1<<8))

// GPIOs des capteurs LIDAR
const uint lidar_pins[NUM_LIDARS] = {3, 4, 5, 6, 7, 8};

// Initialisation des broches en entrée
void initLidarPins(void) {
    for (int i = 0; i < NUM_LIDARS; i++) {
        gpio_init(lidar_pins[i]);
        gpio_set_dir(lidar_pins[i], GPIO_IN); // la configure en entrée
    }
}

/// Retourne un octet dont les 6 bits de poids faible
/// correspondent à GP3..GP8
uint8_t getLidarBits(void) {
    uint32_t all = gpio_get_all(); // lit GP0..GP29 en un mot
    return (uint8_t)((all & LIDAR_MASK) >> 3);
}

int main(void) {
    stdio_init_all(); // USB-série
    initLidarPins();

    while (1) {
        uint8_t bitLidars = getLidarBits();
        printf("LIDAR bits: 0x%02X\n", bitLidars);
    }
    return 0;
}
```

nom / prénom : MARIANA MAKAYA

Principe:

- récupération des données (ligne noir ligne blanche)
- ce code est là pour **transformer la réflectance** sous chaque capteur en une **durée mesurable** (μs), puis, en comparant cette durée à un **seuil prédéfini**, déterminer :

si le capteur est au-dessus d'une **surface blanche** (temps court),
ou d'une **ligne noire** (temps long).

exemple d'utilisation :

Capteur gauche: 62 us → BLANC		Capteur droit : 89 us → BLANC
Capteur gauche: 58 us → BLANC		Capteur droit : 88 us → BLANC
Capteur gauche: 587 us → NOIR		Capteur droit : 83 us → BLANC
Capteur gauche: 640 us → NOIR		Capteur droit : 97 us → BLANC
Capteur gauche: 814 us → NOIR		Capteur droit : 98 us → BLANC
Capteur gauche: 1326 us → NOIR		Capteur droit : 120 us → BLANC
Capteur gauche: 2827 us → NOIR		Capteur droit : 180 us → BLANC
Capteur gauche: 3107 us → NOIR		Capteur droit : 290 us → BLANC
Capteur gauche: 2408 us → NOIR		Capteur droit : 671 us → NOIR

```
// rc_timing_qre_dual_structured.c
// Version structurée : init vs acquisition

#include "pico/stdlib.h"
#include <stdio.h>
#include "hardware/gpio.h"
#include "hardware/timer.h"

// Broches des capteurs
#define PIN_QRE1      17    // GPIO17 (phys. pin 22)
#define PIN_QRE2      16    // GPIO16 (phys. pin 21)

// Durée de charge du condensateur en microsecondes
#define CHARGE_TIME_US 10

// Seuil pour distinguer BLANC / NOIR
#define THRESHOLD_US   500

// --- Fonctions utilitaires ---

// Mesure du temps de décharge RC pour une broche donnée
static uint32_t measure_rc_time(uint pin) {
    // Charger le condensateur
    gpio_set_dir(pin, GPIO_OUT);
    gpio_put(pin, 1);
    busy_wait_us(CHARGE_TIME_US);

    // Passer en entrée pour laisser décharger
    gpio_set_dir(pin, GPIO_IN);
    gpio_disable_pulls(pin);

    // Chrono jusqu'à bascule à 0
    absolute_time_t start = get_absolute_time();
    while (gpio_get(pin)) {}
    return absolute_time_diff_us(start, get_absolute_time());
}
```

```

// Interprétation du résultat
static const char* interpret(uint32_t t) {
    return (t > THRESHOLD_US) ? "NOIR" : "BLANC";
}

// --- Partie 1 : Initialisation ---

static void init_sensors(void) {
    // Initialise la console USB (printf → UART)
    stdio_init_all();

    // Initialise les GPIO (mode "défaut")
    gpio_init(PIN_QRE1);
    gpio_init(PIN_QRE2);
}

// --- Partie 2 : Acquisition et affichage ---

static void read_and_display(void) {
    uint32_t t1 = measure_rc_time(PIN_QRE1);
    uint32_t t2 = measure_rc_time(PIN_QRE2);

    printf(
        "Capteur 1 : %4u us → %s |      "
        "Capteur 2 : %4u us → %s\r\n",
        t1, interpret(t1),
        t2, interpret(t2)
    );
}

int main() {
    init_sensors();

    while (true) {
        read_and_display();
        sleep_ms(100);
    }

    return 0;
}

```

Partie récupération des données:

```

// Mesure du temps de décharge RC pour une broche donnée
static uint32_t measure_rc_time(uint pin) {
    // 1) Charger le condensateur : config en sortie à HIGH
    gpio_set_dir(pin, GPIO_OUT);
    gpio_put(pin, 1);
    busy_wait_us(CHARGE_TIME_US);
}

```

```

// 2) Passage en entrée, désactivation des pulls
gpio_set_dir(pin, GPIO_IN);
gpio_disable_pulls(pin);

// 3) Démarrage du chronomètre
absolute_time_t start = get_absolute_time();

// 4) Attente de la bascule à 0
while (gpio_get(pin)) {}

// 5) Retour du temps écoulé (µs)
return absolute_time_diff_us(start, get_absolute_time());
}

// Lecture des deux capteurs et affichage temps + BLANC/NOIR
static void read_and_display(void) {
    uint32_t t1 = measure_rc_time(PIN_QRE1);
    uint32_t t2 = measure_rc_time(PIN_QRE2);

    // Interprétation selon le seuil
    const char* s1 = (t1 > THRESHOLD_US) ? "NOIR" : "BLANC";
    const char* s2 = (t2 > THRESHOLD_US) ? "NOIR" : "BLANC";

    // Affichage final
    printf("Capteur 1 : %4u us → %s | Capteur 2 : %4u us → %s\r\n",
        t1, s1,
        t2, s2);
}

```

ABDALLAH BEN SAINDOU

FONCTION : `getBpStart`

◆ Code 1 – Clignotement de LED avec détection de bouton + initialisations

Ce programme utilise un **bouton poussoir connecté au GPIO21** du Raspberry Pi Pico pour **activer ou désactiver un clignotement de la LED intégrée** (GPIO25).

Lorsque l'utilisateur appuie sur le bouton (détection d'un front descendant), plusieurs fonctions d'initialisation simulées (`i2cInit`, `uartInit`, etc.) sont appelées, représentant la configuration de différents modules (capteurs, communication, etc.).

Le clignotement de la LED indique visuellement que les modules sont prêts. Un deuxième appui sur le bouton stoppe le clignotement. Le bouton utilise une résistance de **pull-up interne** pour garantir une lecture stable.

```
#include "pico/stdlib.h" // Bibliothèque standard pour Raspberry Pi Pico
#include <stdio.h>        // Pour fonctions d'entrée/sortie (printf, etc.)

// Définition des broches utilisées
#define LED_PIN 25        // GPIO25 correspond à la LED intégrée sur le Pico
#define BP_PIN 21         // GPIO21 est utilisé pour le bouton-poussoir (BP)

// Variables globales
bool isBlinking = false;  // Indique si le clignotement est actif
bool lastButtonState = true; // Mémoire l'état précédent du bouton

// Fonctions d'initialisation simulées (juste des printf pour démo)
void i2cInit() { printf("i2cInit called\n"); }
```

```

void uartInit() { printf("uartInit called\n"); }
void lidarInit() { printf("lidarInit called\n"); }
void ligneInit() { printf("ligneInit called\n"); }
void initMpu() { printf("initMpu called\n"); }
void initPixy() { printf("initPixy called\n"); }

/**
 * Vérifie si le bouton a été pressé (front descendant)
 * et inverse l'état de clignotement en conséquence.
 * Retourne l'état actuel de clignotement.
 */
bool getBpStart() {
    bool currentButtonState = gpio_get(BP_PIN); // Lecture de l'état actuel du bouton

    // Détection d'un front descendant (passage de 1 à 0)
    if (lastButtonState && !currentButtonState) {
        isBlinking = !isBlinking; // Inverse l'état de clignotement
        sleep_ms(50);             // Pause courte pour éviter les rebonds du bouton

        // Appel des fonctions d'initialisation (simulées ici)
        i2cInit();
        uartInit();
        lidarInit();
        ligneInit();
        initMpu();
        initPixy();
    }

    lastButtonState = currentButtonState; // Met à jour l'état précédent du bouton
    return isBlinking; // Retourne l'état du clignotement
}

int main() {
    stdio_init_all(); // Initialisation de l'interface standard (USB debug)

    // Configuration de la LED
    gpio_init(LED_PIN); // Initialise la broche de la LED
    gpio_set_dir(LED_PIN, GPIO_OUT); // Configure la LED en sortie

    // Configuration du bouton
    gpio_init(BP_PIN); // Initialise la broche du bouton
    gpio_set_dir(BP_PIN, GPIO_IN); // Configure le bouton en entrée
    gpio_pull_up(BP_PIN); // Active la résistance de pull-up interne

    // Boucle principale
    while (true) {
        if (getBpStart()) {
            // Si clignotement activé, faire clignoter la LED

```



```

        gpio_put(LED_PIN, 1); // LED ON
        sleep_ms(500);        // Attente 500 ms
        gpio_put(LED_PIN, 0); // LED OFF
        sleep_ms(500);        // Attente 500 ms
    } else {
        // Sinon, LED éteinte
        gpio_put(LED_PIN, 0); // LED OFF
        sleep_ms(100);        // Pause courte pour éviter la surcharge CPU
    }
}
}
}

```

◆ Code 2 – Lecture simple de bouton avec affichage via USB

Ce programme est un exemple simple pour lire l'état d'un **bouton poussoir connecté au GPIO21** du Raspberry Pi Pico.

Le bouton est configuré en **entrée avec une résistance de pull-up interne**, ce qui signifie qu'il retourne **1 (HIGH)** quand il n'est pas appuyé, et **0 (LOW)** lorsqu'il est pressé.

L'état du bouton est affiché toutes les 100 ms via l'interface USB, ce qui permet de vérifier son fonctionnement en temps réel à l'aide d'un terminal série comme PuTTY, Thonny, ou minicom.

```

#include "pico/stdlib.h" // Bibliothèque standard pour Raspberry Pi Pico
#include <stdio.h>        // Pour utiliser printf et autres E/S standard

#define BP_PIN 21 // Définition du numéro de la broche pour le bouton (GPIO21)

/**
 * Fonction qui lit l'état du bouton.
 * Comme un pull-up est activé, un appui sur le bouton met la ligne à LOW (0).
 * Retourne true (1) si NON appuyé, false (0) si appuyé.
 */
bool getBpStart() {
    return gpio_get(BP_PIN); // Lecture de l'état logique de la broche
    // gpio_get retourne 1 si bouton relâché, 0 si appuyé
}

int main() {
    stdio_init_all(); // Initialise les E/S standard (USB série) pour printf

    gpio_init(BP_PIN); // Initialise la broche du bouton
    gpio_set_dir(BP_PIN, GPIO_IN); // Configure la broche en mode entrée
    gpio_pull_up(BP_PIN); // Active la résistance de pull-up interne

    // Boucle principale
    while (true) {
        bool bp = getBpStart(); // Lit l'état du bouton
    }
}

```

```
    printf("%d\n", bp ? 1 : 0);          // Affiche 1 si bouton relâché, 0 si
appuyé
    sleep_ms(100);                      // Pause de 100 ms entre chaque lecture
}
```